OpenCode + Qwen3 Coder 30B Workflow Report

Executive Summary

Qwen3 Coder 30B is useful as a low-cost local implementation worker when the task is small, bounded, and mechanically verifiable. It should not own architecture, roadmap decisions, security-sensitive flows, Stripe/payment design, or broad refactors.

The best workflow is a control-tower model:

- Cloud control defines scope, writes the handoff, reviews the result, and decides the next step.
- OpenCode/Qwen executes one bounded task at a time.
- Every completed task must pass validation and end in a small git commit.
- If validation fails, the next task is a repair task.

This setup reduces cloud token usage while keeping quality gates strong.

Roles

Cloud Control

Use cloud control for:

- roadmap ownership
- task decomposition
- security and payment architecture
- checkout/order consistency
- reviewing local worker output
- writing exact handoffs
- deciding whether to increase or reduce task scope

Cloud control is the brain. It should be skeptical, precise, and protective of the repository state.

Local Worker

Use Qwen3 Coder 30B for:

- one function rewrites
- one component changes
- small multi-file vertical slices
- read-only inventories

- mechanical API normalization
- low-risk UI wiring
- validation runs

The local worker is the right hand. It can move fast, but it needs rails.

Current Model Assessment

Qwen3 Coder 30B has been competent when:

- the handoff names exact files
- the task has a single objective
- forbidden files are explicit
- validation commands are provided
- the stop condition is clear
- commit instructions are concrete

Observed strengths:

- understands React component structure better than GPT OSS 20B
- handles small frontend refactors well
- can perform useful read-only inventories
- follows file scope better after repeated protocol reinforcement

Observed weaknesses:

- may overclaim validation or completion
- can get confused by PowerShell working directories
- may fail Git operations if not started in repo root
- can hallucinate missing backend checks during read-only analysis
- may update logs or plans ahead of real code state

Conclusion: use it for bounded implementation, not unchecked project ownership.

Standard Work Loop

1. Cloud control reviews current repo state.
2. Cloud control writes or updates `01-HANDOFF.md`.
3. User opens a fresh OpenCode conversation for the task.
4. User pastes the compact task prompt.
5. Local worker reads `01-HANDOFF.md` and `03-WORKER-PROTOCOL.md`.

6. Local worker executes only the current task.
7. Local worker validates.
8. Local worker commits only if validation passes.
9. User shows output or commit hash to cloud control.
10. Cloud control reviews diff and validation.
11. If good, move to next task.
12. If bad, create a repair task before continuing.

Task Size Ladder

Increase scope only when the previous level is reliable.

Level 1: Single Function

Examples:

- rewrite `updateItemQty`
- restore `clearCart`
- fix one token read

Use when the file is broken or the model is unstable.

Level 2: Single File

Examples:

- normalize one API consumer
- clean one component
- add one missing state branch

Use after Level 1 is passing.

Level 3: Small Vertical Slice

Examples:

- add `ProductEdit.jsx`
- add one route in `App.jsx`
- make `AdminProductForm.jsx` support edit mode

This is the current target scope for Qwen3 Coder 30B.

Level 4: Feature Segment

Examples:

- admin product create/edit/delete flow
- profile page plus route guard
- order history page

Only use after multiple Level 3 tasks succeed cleanly.

Level 5: Architecture

Do not delegate this to the local worker.

Examples:

- Stripe PaymentIntent/order lifecycle
- auth token lifecycle
- backend authorization model
- database schema design

Cloud control should design these first, then split into lower-level tasks.

Handoff Standard

Every handoff should contain:

- Task ID
- Task name
- repository path
- branch name
- files to read first
- allowed files
- read-only reference files
- forbidden files
- objective
- exact required behavior
- validation commands
- log entry rules
- commit rule
- stop conditions

The handoff must tell the worker what not to do. This matters as much as the actual objective.

Prompt Template For OpenCode

Paste this at the start of a local worker conversation:

```
Read:
C:\Users\adria\Programacion\Programacion\koala-tshirts\01-HANDOFF.md
C:\Users\adria\Programacion\Programacion\koala-tshirts\03-WORKER-PROTOCOL.md

Execute only the current handoff.

Important:
- Start by confirming the repository root with git rev-parse --show-toplevel.
- If git rev-parse fails, stop and ask me to reopen OpenCode in the repo root.
- Modify only the Allowed Files listed in 01-HANDOFF.md.
- Do not modify forbidden files.
- Do not continue to any other task.
- Run the validation commands from the exact directory specified in 01-HANDOFF.md.
- If validation fails, update the log as failed-validation and stop.
- If validation passes, stage only the allowed changed files and create the task commit.
- Report the commit hash and stop.
```

Git Standard

The local worker must verify location before editing:

```
git rev-parse --show-toplevel
git status --short
```

Expected root:

```
C:\Users\adria\Programacion\Programacion\koala-tshirts
```

If the worker is not in the repository root, it should run:

```
Set-Location C:\Users\adria\Programacion\Programacion\koala-tshirts
```

Then re-check:

```
git rev-parse --show-toplevel
git status --short
```

After validation passes:

```
git add -- <allowed-file-1> <allowed-file-2>
git status --short
git commit -m "T6.4 add admin product edit route"
git rev-parse --short HEAD
```

Rules:

- never stage all files with `git add .`
- never commit if validation failed
- never commit files outside the allowed list
- never mark roadmap tasks done unless the handoff explicitly says so
- if Git fails twice, stop and report the exact failure

Validation Standard

Frontend validation usually means:

```
Set-Location C:\Users\adria\Programacion\Programacion\koala-tshirts\frontend
npm.cmd run build
npm.cmd run lint
```

Backend validation depends on the task. Prefer focused commands first.

Validation results must be factual:

- command run
- directory used
- pass/fail
- exact error summary if failed

Do not accept "it should work" as validation.

Logging Standard

Use `02-LOG.md` as the factual history.

Good log entry:

```
## L15
- task: `T6.4`
- result: `done`
- files: `frontend/src/App.jsx`, `frontend/src/pages/admin/ProductEdit.jsx`, `frontend/src/components/AdminProductForm.jsx`, `02
- validation: `npm.cmd run build` passed; `npm.cmd run lint` passed with existing warnings only
- notes: added admin edit route, edit page, and shared create/edit product form behavior
```

Failed log entry:

```
## L15
- task: `T6.4`
- result: `failed-validation`
- files: `frontend/src/App.jsx`, `frontend/src/pages/admin/ProductEdit.jsx`, `frontend/src/components/AdminProductForm.jsx`, `02
- validation: `npm.cmd run build` failed
- notes: build failed in AdminProductForm.jsx because ...
```

Logs must not claim work that did not pass validation.

Review Checklist

Cloud control should review:

- Did only allowed files change?
- Did the worker modify roadmap state incorrectly?
- Does the diff match the task?

- Did validation really run?
- Are there new build or lint errors?
- Did the worker commit only scoped files?
- Did the implementation preserve existing behavior?
- Does the next recommended task follow from real repo state?

Failure Handling

If the local worker breaks the repo:

1. Stop all feature work.
2. Do not continue roadmap tasks.
3. Create a repair handoff.
4. Repair build/lint first.
5. Correct false log or plan entries.
6. Commit the repair.
7. Resume from the last healthy checkpoint.

The worker should never "fix forward" into unrelated areas.

Recommended Agent Usage In OpenCode

Use planning mode for:

- inventories
- route maps
- identifying boundaries
- creating implementation plans

Use build mode for:

- code edits
- validation
- commits

Do not ask build mode to plan the whole feature. Feed it the plan.

When To Use Cloud Tokens

Use cloud help for:

- Stripe security and order lifecycle

- auth expiry/session design
- database schema or migrations
- production deployment decisions
- reviewing large local-worker diffs
- writing complex handoffs
- resolving repeated local failure loops

Use local Qwen for:

- execution after cloud planning
- mechanical edits
- small UI route wiring
- inventories
- repetitive cleanup

Next Maturity Target

The next target is reliable Level 3 tasks:

- 2-4 files
- one vertical behavior
- build and lint pass
- task commit created
- no unrelated roadmap edits

If Qwen3 Coder 30B completes three Level 3 tasks cleanly, we can try Level 4 feature segments.